

COS30018 - Option C - Task 3 Report

Data processing 2

Student Name: Kha Anh Nguyen

Student ID: 103814796

Date: August 30, 2025

Course: COS30018 - Intelligent Systems

Project: FinTech101 Stock Price Prediction System

Github repo: [Leissha/stock_prediction](https://github.com/Leissha/stock_prediction)

Table of Contents

COS30018 - Option C - Task 3 Report	1
1. Functions design:	1
2. Candlestick Chart Function	2
3. Boxplot Chart Function	4
4. Conclusion	6

1. Functions design:

- I chose Plotly to plot interactive chart with flexible format export options.
- I created a function to plot candlestick chart and another one for boxplot.
- It will be called after raw data loading for exploratory data analysis (EDA):

```
# Create chart for data inspection
try:
    chart_path =
f"dev/cache/inspect_data/{ticker}_{start_date}_to_{end_date}"
    os.makedirs(os.path.dirname(chart_path), exist_ok=True)

    candlestick_path = f"{chart_path}/candlestick_chart.png"
    boxplot_path = f"{chart_path}/boxplot.png"

# Check if chart files exist
if not os.path.exists(candlestick_path):
```

```

        create_candlestick_chart(df, ticker, save_path=candlestick_path,
n_days=lag_days)
        logger.info(f"Candlestick chart saved to: {candlestick_path}")
    else:
        logger.info("Candlestick chart already exists")

    if not os.path.exists(boxplot_path):
        create_boxplot(df, ticker, save_path=boxplot_path,
n_days=lag_days)
        logger.info(f"Boxplot saved to: {boxplot_path}")
    else:
        logger.info("Boxplot already exists")

except Exception as e:
    logger.error(f"Chart creation failed: {e}")

```

```

=====
STOCK PREDICTION PIPELINE
=====
2025-08-30 14:43:33.370 | INFO | __main__:train:79 - === TRAINING PHASE ===
2025-08-30 14:43:33.370 | INFO | data_preprocessing.data_loading:load_stock_data:19 - Loading cached data for CBA.AX
2025-08-30 14:43:33.372 | INFO | data_preprocessing.data_processor:data_processing:88 - Loaded data: (506, 5) from 2023-08-31 to 2025-08-30
2025-08-30 14:43:33.372 | INFO | data_preprocessing.data_processor:data_processing:89 - Available columns: ['Close', 'High', 'Low', 'Open', 'Volume']
2025-08-30 14:43:33.375 | INFO | data_preprocessing.data_processor:data_processing:103 - Creating charts at: dev/cache/inspect_data/CBA.AX_2023-08-31_to_2025-08-30
2025-08-30 14:43:33.375 | INFO | data_preprocessing.data_processor:data_processing:111 - Candlestick chart already exists
2025-08-30 14:43:33.375 | INFO | data_preprocessing.data_processor:data_processing:118 - Boxplot already exists
2025-08-30 14:43:33.375 | INFO | data_preprocessing.data_processor:data_processing:135 - Input features (OHLCV): ['close', 'high', 'low', 'open', 'volume']
2025-08-30 14:43:33.375 | INFO | data_preprocessing.data_processor:data_processing:136 - Target feature: close
2025-08-30 14:43:33.377 | INFO | data_preprocessing.handle_nans:handle_nans:28 - No NaN values detected

```

2. Candlestick Chart Function

- The function below visualizes the dataset as candlestick chart with option to aggregate multiple trading days into a single candle.
- In addition, Plotly allows the user to zoom out and see broader price trends with HTML and export a static PNG.
- I also added SMA (Simple Moving Average) and EMA (Exponential Moving Average) overlays to highlight long-term vs. short-term trends.

```

def create_candlestick_chart(df, ticker,
save_path="inspect_data/candlestick_chart.png", n_days=1):
    """
    Create candlestick chart with SMA and EMA.
    Each candlestick can represent n trading days (aggregated OHLC).
    """
    # Ensure datetime index
    df.index = pd.to_datetime(df.index)

```

```

# Aggregate OHLC values if n_days > 1
if n_days > 1:
    df = df.resample(f'{n_days}D').agg({
        'open': 'first', # first value in the window
        'close': 'last', # last value in the window
        'high': 'max',   # max value in the window
        'low': 'min'     # min value in the window
    }).dropna()

# Add SMA and EMA if they don't exist
if 'SMA' not in df.columns:
    df['SMA'] = df['close'].rolling(window=20, min_periods=1).mean() #
min_periods=1 to handle NaN
if 'EMA' not in df.columns:
    df['EMA'] = df['close'].ewm(span=20, min_periods=1).mean() #
min_periods=1 to handle NaN

# Base candlestick
candlestick = go.Candlestick(
    x=df.index,
    open=df['open'],
    high=df['high'],
    low=df['low'],
    close=df['close'],
    name="Candlestick"
)

# Simple Moving Average (SMA)
sma = go.Scatter(
    x=df.index,
    y=df['SMA'],
    mode='lines',
    line=dict(color='blue'),
    name="SMA (20)"
)

# Exponential Moving Average (EMA)
ema = go.Scatter(
    x=df.index,
    y=df['EMA'],
    mode='lines',
    line=dict(color='orange'),
    name="EMA (20)"
)

```

```

# Build figure
fig = go.Figure(data=[candlestick, sma, ema])
fig.update_layout(
    width=900, height=600,
    title=f"{ticker} Candlestick with SMA & EMA",
    yaxis_title=f"{ticker} Stock Price",
    xaxis_title="Date",
    xaxis_rangeflider_visible=False
)

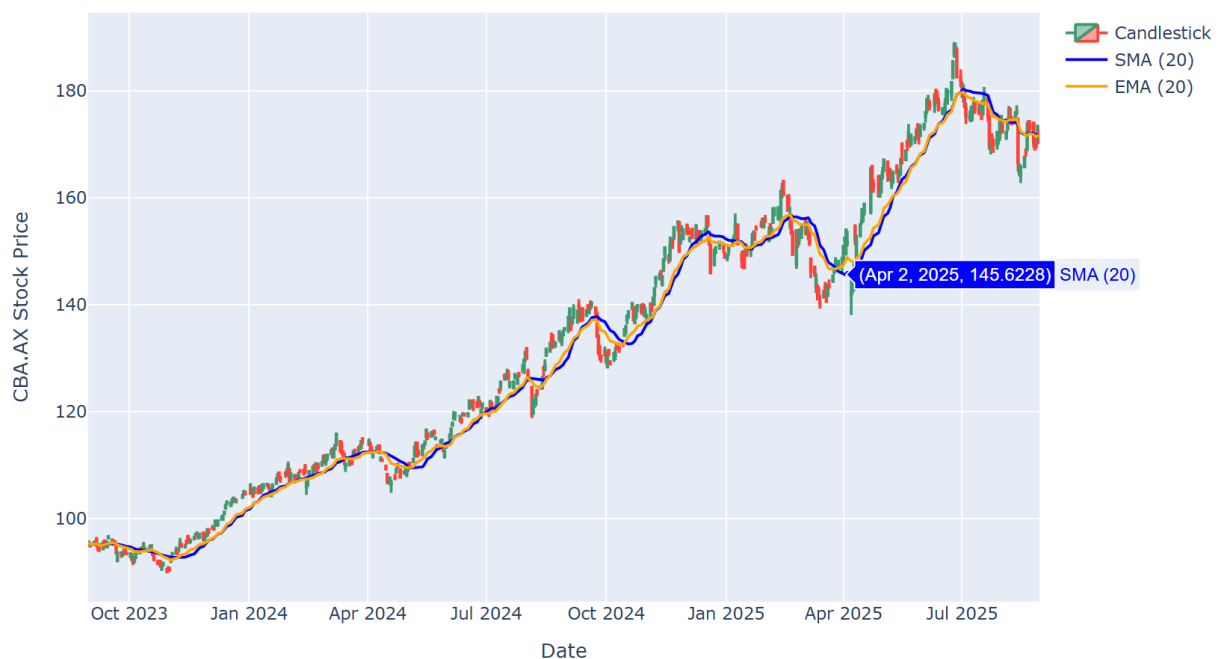
# Ensure save directory exists
os.makedirs(os.path.dirname(save_path), exist_ok=True)

fig.write_image(save_path)
fig.show()

```

Output chart:

CBA.AX Candlestick with SMA & EMA



3. Boxplot Chart Function

- I use boxplots to summarize the distribution of stock closing prices across consecutive n-day windows. This visualization is particularly useful for spotting **volatility**, **skewness**, and **unusual outliers**.

- I set `n_days` to 20, which is the typical trading days in 1 month.

```
def create_boxplot(df, ticker, save_path="inspect_data/boxplot.png", n_days=20):
    """
    Create boxplots of stock closing prices for consecutive n-day windows.
    Shows distribution of prices (median, quartiles, outliers).
    """
    traces = []
    df.index = pd.to_datetime(df.index)

    for i in range(0, len(df) - n_days + 1, n_days):
        window_data = df.iloc[i:i+n_days]['close'].dropna()
        period_name = f"{df.index[i].strftime('%d-%m-%y')} → {df.index[i+n_days-1].strftime('%d-%m-%y')}"

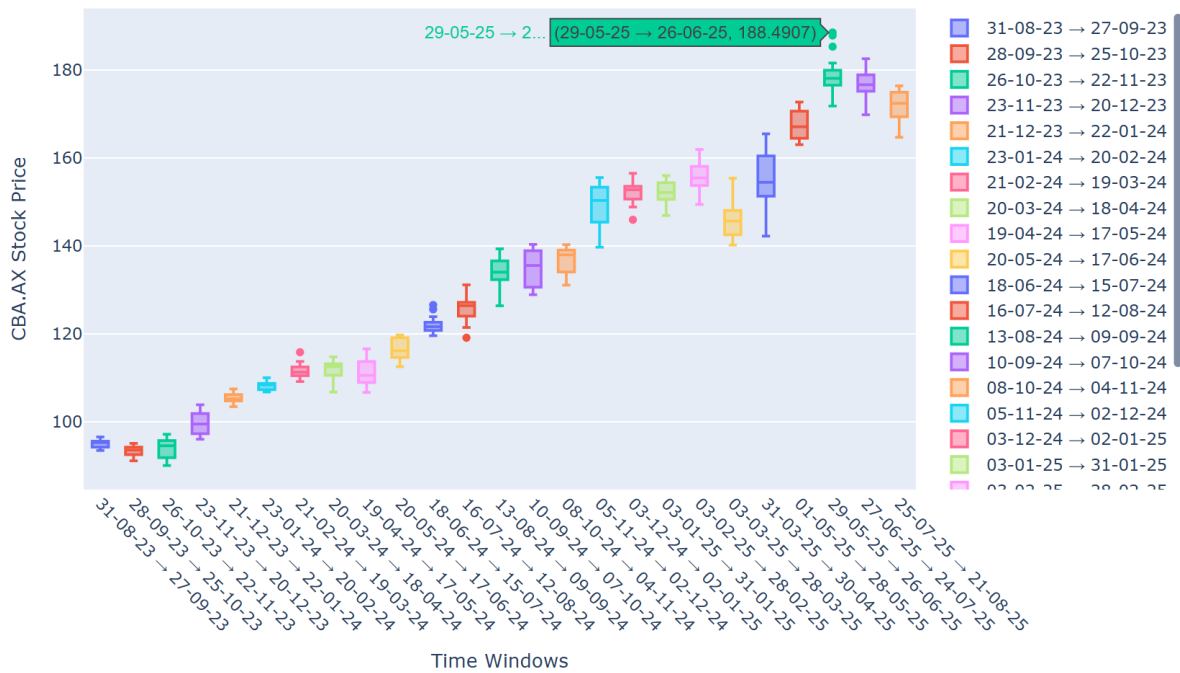
        traces.append(go.Box(
            y=window_data,
            name=period_name,
            boxpoints='outliers'
        ))

    fig = go.Figure(data=traces)
    fig.update_layout(
        width=900, height=600,
        title=f"{ticker} Closing Price Distribution ({n_days}-Trading-Day windows)",
        yaxis_title=f'{ticker} Stock Price',
        xaxis_title="Time Windows",
        xaxis_rangeslider_visible=False,
        xaxis=dict(
            tickangle=45
        )
    )

    fig.write_image(save_path)
    fig.show()
```

Output chart:

CBA.AX Closing Price Distribution (20-Trading-Day windows)



Static PNssssG charts will be saved to cache folder:



4. Conclusion

This task helps me to gain deeper knowledge of how visualization supports exploratory data analysis in stock price prediction. The candlestick chart (with SMA and EMA overlays) highlights **daily price movement** and **long/short-term trends**, while the boxplot

summarizes monthly distributions, making it easier to **spot volatility** and **outliers**.

Together, these charts provide complementary perspectives: *candlesticks capture detailed trading dynamics*, and *boxplots provide a higher-level statistical view*.